

Anticiper la consommation énergétique et les émissions de CO₂ des bâtiments

Projet 6 – Modélisation supervisée & API de prédiction

Seattle • bâtiments non résidentiels





Besoin métier

Seattle souhaite réduire son empreinte carbone et mieux maîtriser les consommations énergétiques des bâtiments.



Problème

Les relevés manuels ou énergétiques sont coûteux : il faut estimer automatiquement à partir de données déjà disponibles.



Solution Data

Construire des modèles supervisés capables de prédire la consommation totale et les émissions de CO₂.

Vous travaillez en tant que Data Engineer pour la **ville de Seattle**. Pour atteindre son objectif de ville neutre en émissions de carbone en 2050, votre équipe s'intéresse de près à la consommation et aux émissions des **bâtiments non destinés à l'habitation**.



Seattle

Des relevés minutieux ont été effectués par les agents de la ville en 2016. Voici [les données](#) et [leur source](#). Ces relevés sont coûteux à obtenir, et à partir de ceux déjà réalisés, vous voulez tenter de **prédire les émissions de CO2** et la **consommation totale d'énergie** de bâtiments **non destinés à l'habitation** pour lesquels elles n'ont pas encore été mesurées.

Problématique et objectifs

03

Deux cibles métier complémentaires

Consommation énergétique

Cible : SiteEnergyUse(kBtu)

Objectif : estimer la consommation totale d'énergie d'un bâtiment à partir de ses caractéristiques.

Émissions de CO₂

Cible : TotalGHGEmissions

Objectif : estimer l'impact environnemental associé au bâtiment.

Principe clé : utiliser uniquement des variables connues avant la mesure réelle pour éviter toute fuite de données.

Données utilisées : bâtiments de Seattle

Une ligne = un bâtiment

- Surface totale et surfaces par usage
- Usage principal du bâtiment
- Année de construction et âge
- Localisation et caractéristiques structurelles
- Variables mesurées : énergie et émissions

Données riches : structure + usage + localisation + indicateurs énergétiques.

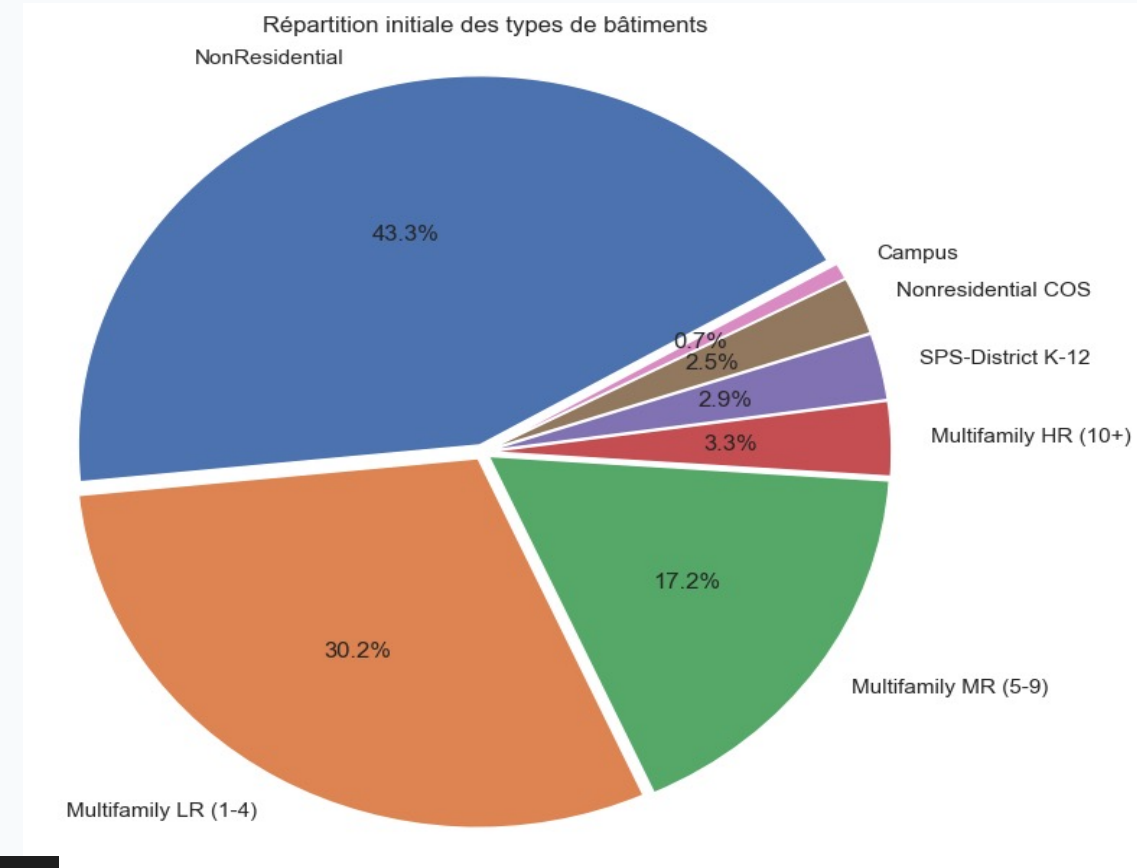
Shape initiale : (3376, 46)

Types de variables :
float64 22
str 14
int64 8
object 1
bool 1
Name: count, dtype: int64

	OSEBuildingID	DataYear	BuildingType	PrimaryPropertyType	PropertyName	Address	City	State	ZipCode	TaxParcelIdentificationNumber	Count
0	1	2016	NonResidential	Hotel	Mayflower park hotel	405 Olive way	Seattle	WA	98101.0	0659000030	1
1	2	2016	NonResidential	Hotel	Paramount Hotel	724 Pine street	Seattle	WA	98101.0	0659000220	1
2	3	2016	NonResidential	Hotel	5673-The Westin Seattle	1900 5th Avenue	Seattle	WA	98101.0	0659000475	1
3	5	2016	NonResidential	Hotel	HOTEL MAX	620 STEWART ST	Seattle	WA	98101.0	0659000640	1
4	8	2016	NonResidential	Hotel	WARWICK SEATTLE HOTEL (ID8)	401 LENORA ST	Seattle	WA	98121.0	0659000970	1

Traitements de périmètre

- Identifier les types de bâtiments
- Exclure les bâtiments résidentiels ou hors périmètre
- Conserver les bâtiments cohérents avec l'objectif métier
- Documenter les lignes supprimées et la justification



```
... Nombre de lignes avant filtrage : 3376
Nombre de lignes après filtrage : 1656
Nombre de lignes supprimées : 1720
Pourcentage supprimé : 50.95%
Types restants : <StringArray>
[ 'NonResidential', 'Nonresidential COS', 'SPS-District K-12',
  'Campus', 'Nonresidential WA']
Length: 5, dtype: str
```

Nettoyage : fiabiliser les données avant modélisation

06



```
Shape initiale : (3376, 46)
```

```
Types de variables :
```

```
float64    22
```

```
str        14
```

```
int64       8
```

```
object      1
```

```
bool        1
```

```
Name: count, dtype: int64
```

```
Nombre de colonnes avant suppression : 42
```

```
Nombre de colonnes après suppression : 16
```

```
Nombre de colonnes supprimées : 26
```

```
Shape après suppression des colonnes : (1656, 16)
```

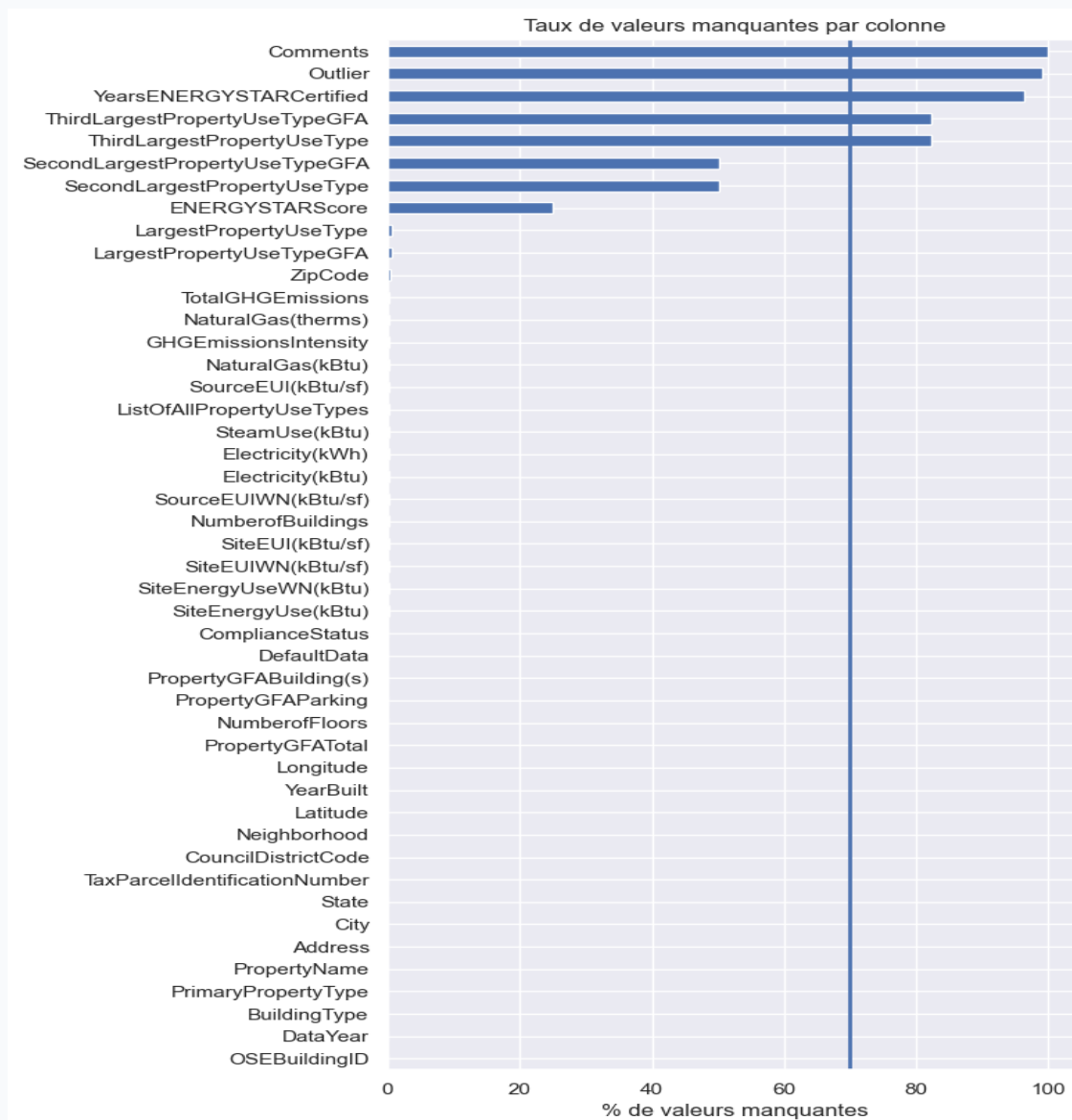
Capture notebook : shape avant / après nettoyage

Important : ne pas supprimer automatiquement toutes les lignes avec valeurs manquantes.

Valeurs manquantes et incohérences

07

Une approche ciblée plutôt qu'une suppression massive



1. Supprimer

Colonnes trop incomplètes ou non exploitables.

2. Imputer

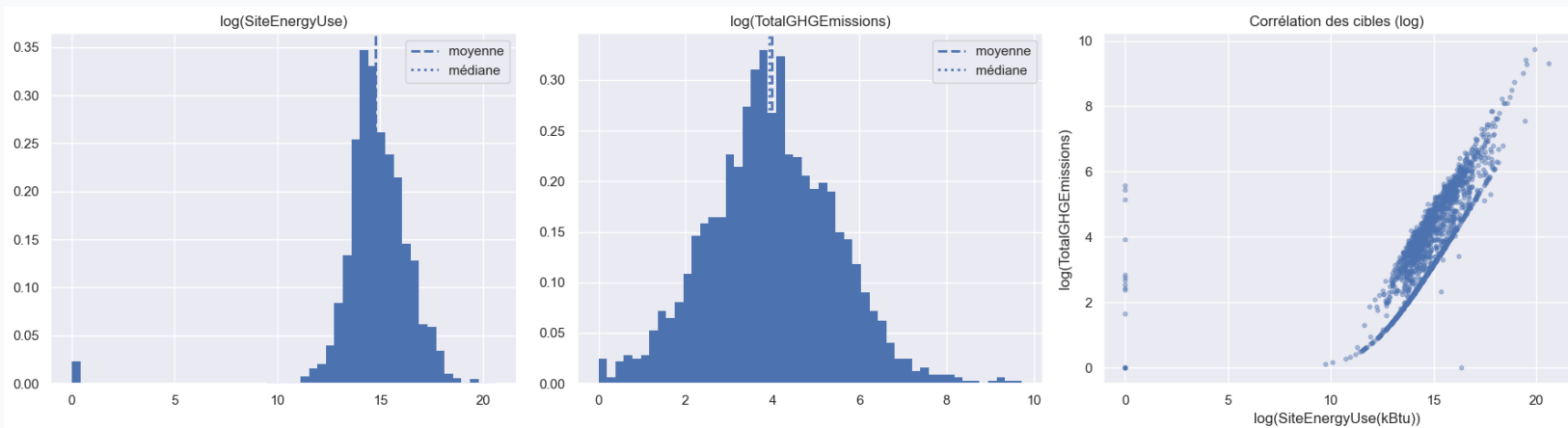
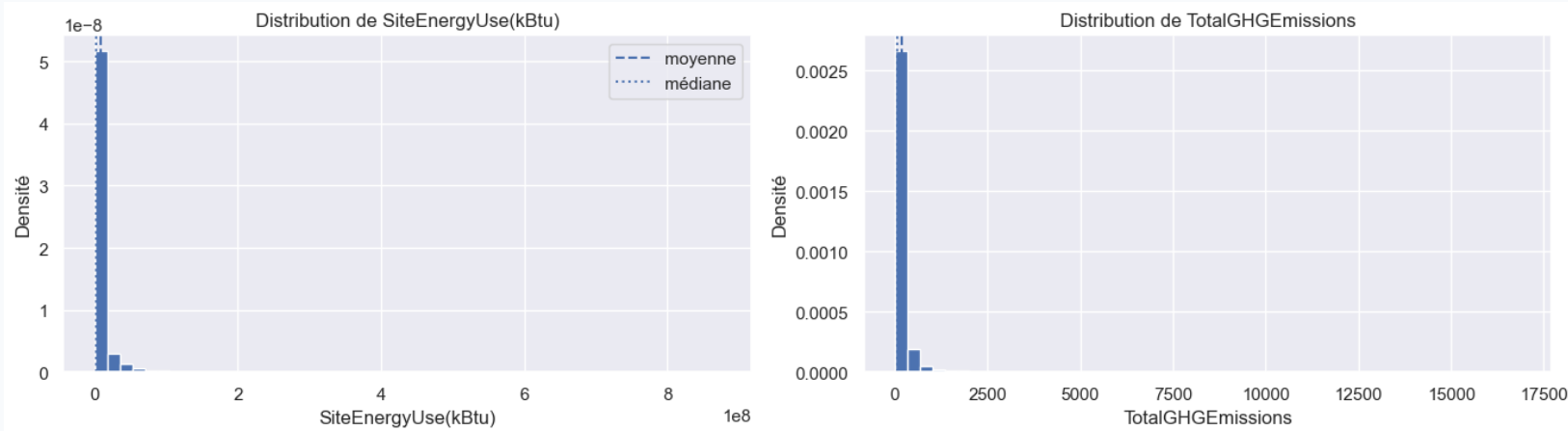
Variables utiles conservées avec traitement adapté.

3. Remplacer logiquement

Lorsque l'absence a un sens métier.

Objectif

Conserver un volume suffisant de données fiables.



Constats principaux

- Distributions très asymétriques
- Quelques bâtiments très consommateurs
- Surface fortement liée aux cibles
- Usage du bâtiment déterminant
- Log-transform utile pour stabiliser

Feature / traitement	Intérêt métier
Âge du bâtiment	tenir compte de l'ancienneté
Catégories rares	réduire la complexité
Indicateur multi-usage	représenter la diversité d'usage
Surfaces par usage	mieux expliquer les besoins
Encodage catégories	rendre les données utilisables

⚠ Data leakage

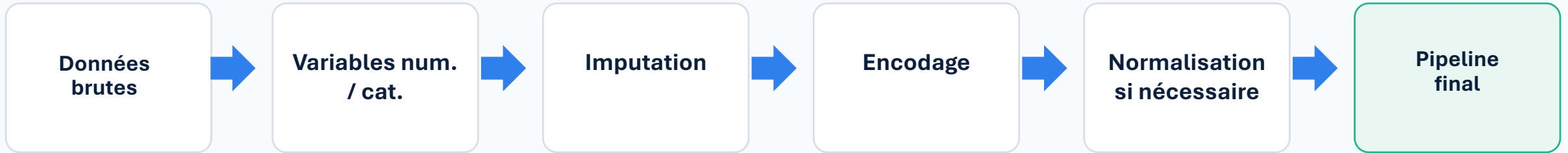
Variables exclues : consommations d'électricité, gaz, émissions mesurées et toute variable calculée à partir de la cible.

Capture optionnelle : création d'une feature

Ex. âge du bâtiment

Préparation des features pour les modèles

10



Capture code : ColumnTransformer / OneHotEncoder / Pipeline

```
# Prétraitement des variables numériques
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# Prétraitement des variables catégorielles
# handle_unknown='ignore' permet de gérer une catégorie absente du train mais
# présente dans le test.
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='constant', fill_value='Unknown')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', max_categories=20))
])

# Préprocesseur global
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ]
)
```

Le pipeline garantit les mêmes transformations à l'entraînement, au test et dans l'API.

Protocole de modélisation : comparer plusieurs modèles

11



Protocole

- séparation train / test
- validation croisée
- mêmes métriques
- mêmes jeux de données
- random state fixé



Modèles testés

- Ridge / ElasticNet
- Régression linéaire
- Random Forest
- Gradient Boosting
- XGBoost ou équivalent



Métriques

- R^2 : variabilité expliquée
- MAE : erreur moyenne lisible
- RMSE : pénalise les grosses erreurs

```
def evaluate_model_cv(model_name, model, X_train, y_train, preprocessor, cv=5):  
  
    pipeline = Pipeline(steps=[  
        ('preprocessor', preprocessor),  
        ('model', model)  
    ])  
  
    scoring = {  
        'r2': 'r2',  
        'mae': 'neg_mean_absolute_error',  
        'rmse': 'neg_root_mean_squared_error'  
    }  
  
    cv_results = cross_validate(  
        pipeline,  
        X_train,  
        y_train,
```

```
...    Nombre de modèles à comparer : 7  
        - DummyRegressor  
        - LinearRegression  
        - Ridge  
        - Lasso  
        - ElasticNet  
        - RandomForest  
        - GradientBoosting
```

Captures: extrait de la fonction d'évaluation / liste des modèles

Comparaison des modèles sans transformation log

12

Les modèles sont d'abord évalués sur la cible originale **SiteEnergyUse(kBtu)**, sans transformation logarithmique.

Modèle	R ² moyen	MAE moyen	RMSE moyen	Commentaire
ElasticNet	0.519	5,342,821	17,489,057	Meilleur R ² et RMSE
Ridge	0.428	6,338,890	19,052,249	Modèle linéaire régularisé
Lasso	0.402	6,487,458	19,679,553	Proche de la régression linéaire
LinearRegression	0.402	6,498,020	19,692,326	Baseline linéaire
RandomForest	0.410	4,767,684	22,144,729	Meilleure MAE mais forte variance
GradientBoosting	0.310	5,150,534	23,564,983	Moins performant sans log
DummyRegressor	-0.073	7,504,311	29,473,843	Modèle de référence naïf

- **ElasticNet obtient le meilleur R² moyen : 0.519**
- **RandomForest obtient la meilleure MAE**
- Les modèles non linéaires sont moins stables sur la cible brute
- La forte dispersion de la consommation rend la prédiction difficile

Cette première comparaison montre les limites de la cible brute : les erreurs restent élevées et les modèles sont sensibles aux valeurs extrêmes.

Comparaison des modèles avec transformation log1p

13

Les modèles sont d'abord évalués sur la cible originale **SiteEnergyUse(kBtu)**, avec transformation logarithmique.

Modèle	R ² train	R ² moyen	MAE moyen	RMSE moyen	Commentaire
GradientBoosting	0.736	0.462	4,398,783	22,237,960	Meilleur compromis avec log
RandomForest	0.768	0.439	4,656,510	22,488,420	Bon résultat, léger surapprentissage
Lasso	-0.037	-0.070	7,505,154	29,447,970	Peu adapté avec log
DummyRegressor	-0.038	-0.073	7,504,311	29,473,840	Référence naïve
ElasticNet	-0.007	Très négatif	Très élevé	Très élevé	Instable avec log
LinearRegression	-12.172	Très négatif	Très élevé	Très élevé	Instable avec log
Ridge	-13.995	Très négatif	Très élevé	Très élevé	Instable avec log

- Le **GradientBoosting** devient le meilleur modèle avec log1p
- La **MAE diminue fortement** par rapport aux modèles sans log
- RandomForest reste performant mais montre un écart train/test
- Les modèles linéaires deviennent instables après transformation inverse

Avec log1p, **Gradient Boosting** devient le meilleur compromis entre performance et stabilité.



Optimisation

GridSearchCV ou recherche d'hyperparamètres : petite grille puis grille élargie.



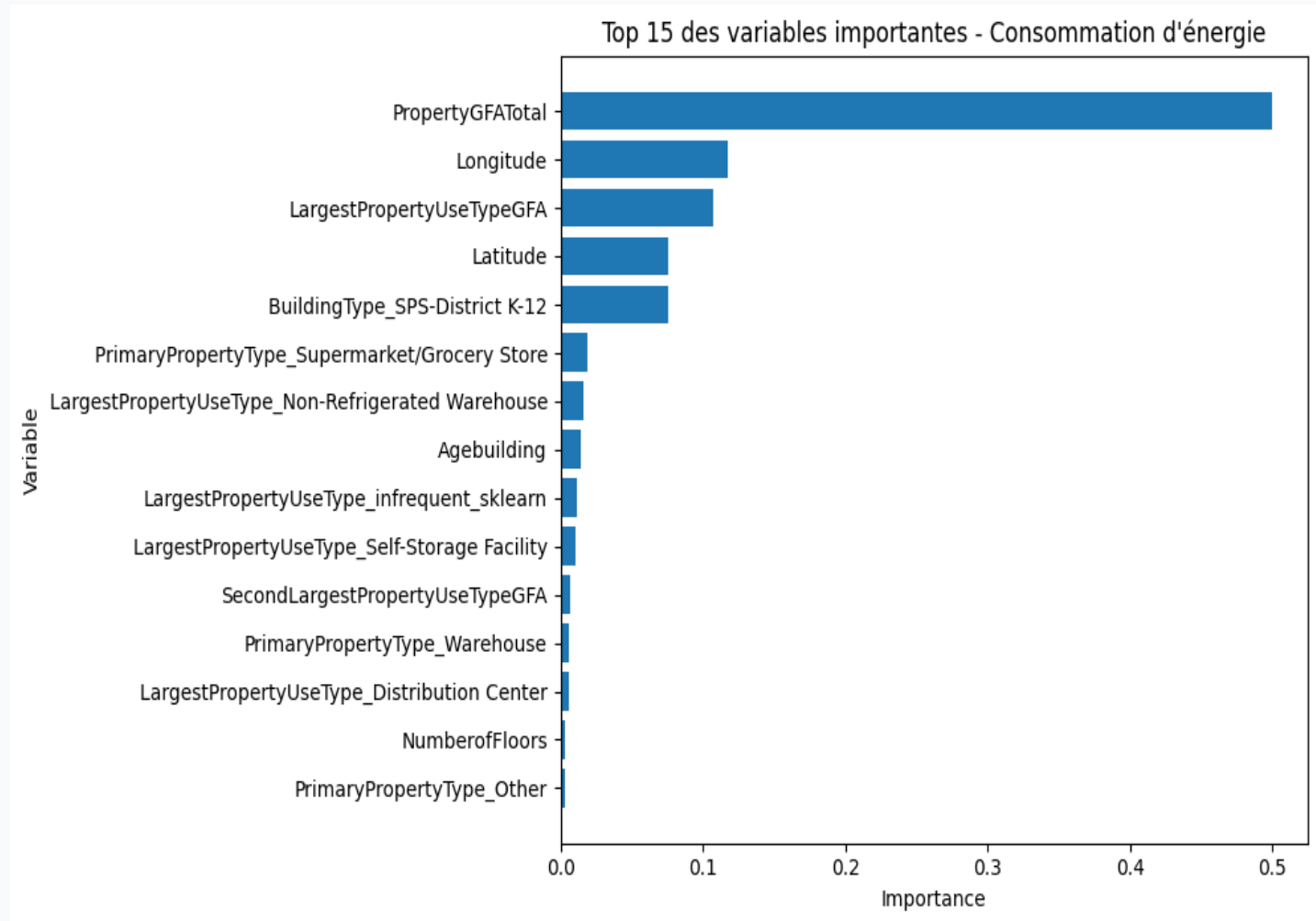
Interprétation

Feature importances pour comprendre les variables les plus influentes.



Lecture métier

Surface, usage, âge, structure et localisation sont souvent déterminants.



Même démarche appliquée aux deux cibles : optimisation puis interprétation.

ENERGYSTARScore : variable utile ou non ?

Analyse à présenter

L'ENERGYSTARScore apporte peu de gain pour la consommation énergétique, mais améliore davantage la prédiction des émissions de CO₂. Pour l'API, son utilisation dépend donc du compromis entre performance et disponibilité de cette variable.

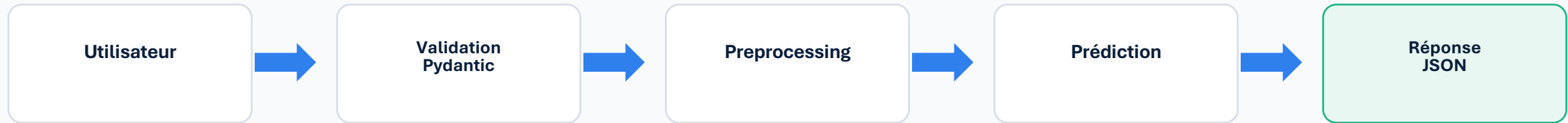
Cible	Sans ENERGYSTARScore	Avec ENERGYSTARScore	Impact
SiteEnergyUse(kBtu) — R ²	0.665	0.675	+0.010
SiteEnergyUse(kBtu) — MAE	3,500,933	3,257,925	amélioration
TotalGHGEmissions — R ²	0.714	0.854	+0.140
TotalGHGEmissions — MAE	92.08	78.07	amélioration

	dataset_version	target	model	target_transformation	R2_test	MAE_test	RMSE_test
0	without_ENERGYSTARScore	SiteEnergyUse(kBtu)	GradientBoosting	log1p	0.665069	3.500933e+06	9.767012e+06
1	without_ENERGYSTARScore	TotalGHGEmissions	GradientBoosting	log1p	0.713933	9.208355e+01	2.623681e+02
2	with_ENERGYSTARScore	SiteEnergyUse(kBtu)	GradientBoosting	log1p	0.675131	3.257925e+06	9.619172e+06
3	with_ENERGYSTARScore	TotalGHGEmissions	GradientBoosting	log1p	0.853665	7.806722e+01	1.876511e+02

Conclusion : variable intéressante, mais à discuter selon sa disponibilité au moment de l'inférence.

Exposition du modèle via une API BentoML

16



Capture service.py : endpoint /predict

```
1 import bentoml
2 import pandas as pd
3 from pydantic import BaseModel, Field
4
5
6 MODEL_TAG = "building_energy_model:buwe3fc7d6v7ldef"
7
8 model_ref = bentoml.sklearn.get(MODEL_TAG)
9 model = model_ref.load_model()
10
11 feature_names = model_ref.custom_objects["feature_names"]
12
```

```
@bentoml.service
class BuildingEnergyService:

    @bentoml.api
    def predict_energy(self, input_data: BuildingInput) -> dict:
        input_df = pd.DataFrame([input_data.model_dump()])

        input_df = input_df[feature_names]

        prediction = model.predict(input_df)[0]

        return {
            "prediction_SiteEnergyUse_kBtu": float(prediction),
            "model": MODEL_TAG
        }
```

BentoML

Pydantic

Capture Swagger

BuildingEnergyService:dev

BentoML 1.4.39 docs passing Join BentoML Slack Stars 8.7k Follow BentoML

This is a Machine Learning Service created with BentoML.

Help

- [Documentation](#): Learn how to use BentoML.
- [Community](#): Join the BentoML Slack community.
- [GitHub Issues](#): Report bugs and feature requests.
- Tip: you can also [customize this README](#).

Contact BentoML Team

Servers

Service APIs

BentoML Service API endpoints for inference.

POST /predict_energy

Response body

```
{
  "prediction_SiteEnergyUse_kBtu": 2753606.7461169735,
  "model": "building_energy_model:buwe3fc7d6v7ldef"
}
```

Response headers

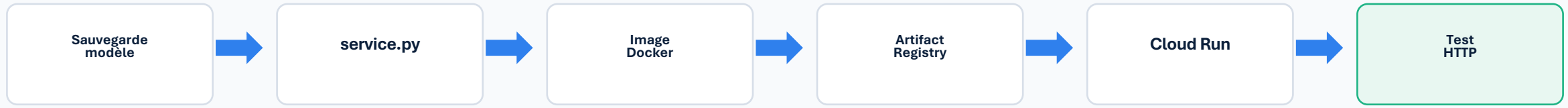
```
content-length: 104
content-type: application/json
date: Wed, 03 Jun 2026 08:37:57 GMT
server: BentoML Service/BuildingEnergyService
x-bentoml-request-id: 570b843f37e35081
```

Docker

Cloud Run

Déploiement du modèle sur Google Cloud Run

17



Endpoint déployé et testé avec succès : réponse JSON contenant une prédiction.

```
(.venv) (base) abdellah@MBP-de-Abdellah Projet_6_Batiment % bentoml containerize building_energy_service:kzkknuc7ekjcldef
/opt/miniconda3/lib/python3.13/site-packages/click/core.py:1193: UserWarning: The parameter --quiet is used more than once. Remove duplicate as parameters should be unique.
  parser = self.make_parser(ctx)
/opt/miniconda3/lib/python3.13/site-packages/click/core.py:1186: UserWarning: The parameter --quiet is used more than once. Remove duplicate as parameters should be unique.
  self.parse_args(ctx, args)
INFO: Building OCI-compliant image for building_energy_service:kzkknuc7ekjcldef with docker

[+] Building 97.5s (17/17) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.2s
=> => transferring dockerfile: 2.84kB                             0.8s
```

```
> => unpacking to docker.io/library/building_energy_service:kzkknuc7ekjcldef 12.0s
Successfully built Bento container for "building_energy_service:kzkknuc7ekjcldef" with tag(s)
"building_energy_service:kzkknuc7ekjcldef"
To run your newly built Bento container, run:
  docker run --rm -p 3000:3000 building_energy_service:kzkknuc7ekjcldef
```

```
Deploying container to Cloud Run service [building-energy-service] in project [oc-projet6-building-energy] region [europe-west1]
✓ Deploying new service... Done.
✓ Creating Revision...
✓ Routing traffic...
✓ Setting IAM Policy...
Done.
Service [building-energy-service] revision [building-energy-service-00001-h9c] has been deployed and is serving traffic.
Service URL: https://building-energy-service-247422449010.europe-west1.run.app
```

```
(.venv) (base) abdellah@MBP-de-Abdellah Projet_6_Batiment % gcloud run deploy building-energy-service \
--image europe-west1-docker.pkg.dev/oc-projet6-building-energy/building-energy-repo/building-energy-service:kzkknuc7ekjcldef \
--platform managed \
--region europe-west1 \
--allow-unauthenticated \
--port 3000 \
--memory 2Gi \
--cpu 1
```

```
(.venv) (base) abdellah@MBP-de-Abdellah Projet_6_Batiment % curl -X POST https://building-energy-service-247422449010.europe-west1.run.app/
{"Latitude": 47.61,
 "Longitude": -122.33,
 "NumberOfBuildings": 1,
 "NumberOfFloors": 3,
 "PropertyGFATotal": 50000,
 "PropertyGFAParking": 0,
 "LargestPropertyUseType": "Office",
 "LargestPropertyUseTypeGFA": 45000,
 "SecondLargestPropertyUseType": "None",
 "SecondLargestPropertyUseTypeGFA": 0,
 "Agebuilding": 40
}
{"prediction_SiteEnergyUse_kBtu": 2753606.7461169735, "model": "building_energy_model:buwe3fc7d6v7ldef"}
```

Exemple : building-energy-service → prédiction SiteEnergyUse(kBtu) selon endpoint.

Limites

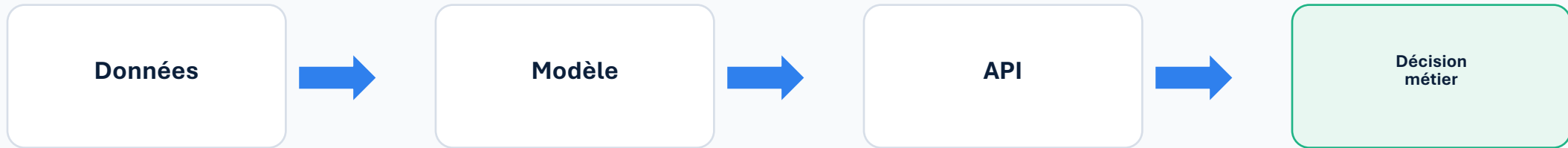
- données limitées à une période donnée
- bâtiments atypiques difficiles à prédire
- variables incomplètes ou bruitées
- modèle dépendant de la qualité des entrées
- estimation, pas remplacement d'un relevé réel

Perspectives

- enrichir avec plusieurs années de données
- ajouter isolation, chauffage, éclairage
- améliorer la sélection de variables
- suivre la performance de l'API
- intégrer un monitoring en production

Conclusion : du notebook au prototype exploitable

19



Données nettoyées et préparées
Deux cibles prédites : énergie et CO₂
Plusieurs modèles comparés et optimisés
Interprétation par feature importances
API déployée pour rendre le modèle utilisable

Le modèle ne remplace pas un relevé réel, mais il fournit une estimation utile pour prioriser les audits énergétiques et identifier les bâtiments à fort impact potentiel.

Merci pour votre attention — Questions ?